

maintains its own fake / mock / virtual DOM in memory where it performs all operations. It then compares this to the actual DOM, and makes the minimal changes required to bring it up to date with the virtual DOM.

By batching these changes React can also ensure that there is no unnecessary rendering of intermediate states that you wouldn't want the user to see anyway.

Setting up a React App

React is composed of two main JavaScript files 'react.js' and 'react-dom.js' that provide the bulk of the functionality. These are all you need to include in your HTML page to use React.

If you want to use the JSX syntax, and want it to automatically work in the browser you need to include the Babel compiler. Your JSX code can be included like any other JavaScript file, with just the type attribute set to "text/babel". This is very useful while testing. We will also include jQuery to simplify communicating with other services.

To quickly get you started, we have created a project with everything set up at <http://dgit.in/ReactTutorial>. You can click on the Remix button to make your own copy of the project and edit it online. You will see a folder for each step of the project with individual HTML and script files. You merely need to edit the "script.js" file. Here you can also click on the "TUTORIAL" tab for more instructions.

Over the rest of this article, we will be building a React application that uses the OMDb (The Open Movie Database) API to search for movies.

Our First Component

The simplest React component need only a method called render that returns the content to display. For example:

```
var Movie = React.createClass({
  render: function() {
    return (
      <div>
        <h3>12 Angry Men</h3>
        <p>Year: 1957</p>
      </div>
    );
  }
});
```

Note how we are directly using an HTML-like syntax to build out UI. We can do this because we are using JSX and the compiler will convert this to pure JavaScript. To see the equivalent JavaScript code, you can type this code into the Babel REPL online at <https://babeljs.io/repl/>. You'll see how better off you are using JSX for React.

The above code alone will show nothing in the browser. It creates a component that subsequently need to create an instance of to display. This is simple enough:

```
ReactDOM.render(<Movie />, document.getElementById('content'));
```

See how we are now using Movie like a tag as if it were any other HTML tag. You can also use this with existing HTML tags like div, span, and a. This is what React is all about. We could also do the above as:

```
var movieInstance = <Movie />;
ReactDOM.render(movieInstance, document.getElementById('content'));
```

The second parameter provided to ReactDOM.render is a direct reference to the DOM node where you wish to inject your component. For this to work there needs to be a tag with an id of 'content' in the

body of your HTML document. The project we have set up for you already has this.

Now you should be seeing something on the screen. Next we'll see how we can supply the movie title and year to a movie object rather than having it hard coded. We can do this with properties.

Component Properties

Component properties allow us to initialise a movie component as follows:

```
<Movie title="12 Angry Men" year={1957} />
```

This is again similar to HTML tags except that we are also using curly braces. These allow you to pass variables, numbers and complex data objects. Here by putting "1957" in curly braces we are passing it to the component as a number rather than a string. In our particular case it doesn't matter though.

Now we can modify the component to use these properties rather than hard-coding them:

```
var Movie = React.createClass({
  render: function() {
    return (
      <div>
        <h3>{this.props.title}</h3>
        <p>Year: {this.props.year}</p>
      </div>
    );
  }
});
```

Any properties you provide are made available via the props object on the component object.

Here you can see how we initialise two different Movie components, and place them in a div:

```
var movies = <div>
  <Movie title="12 Angry Men" year={1957} />
  <Movie title="Donnie Darko" year={2001} />
</div>;
ReactDOM.render(movies, document.getElementById('content'));
```

Properties allow you to pass data and configuration to components but it's very important to note that these properties are never supposed to be modified by the component. React components have a different mechanism, called states that are designed to be modified by the component.

Component State

To show how we can use states, we'll create a simple component that toggles between an empty and a full star. This can be useful to mark something as favourite or important.

This component will need a state called starred is toggled whenever it is clicked. In react the way to do this is to set up the initial state using the `getInitialState` method, and then modify the state using `setState`.

```
var Star = React.createClass({
  getInitialState: function() {
    return {starred: false};
  },
  star: function() {
    return this.state.starred ? "\u2605" : "\u2606";
  },
  handleClick: function() {
    this.setState({starred: !this.state.starred});
  },
});
```